Studio Nx

Backend(FastAPI/Python) 19 — check_email() · availability_of()

2026-10-12

Backend(FastAPI/Python) 19 — `check_email()` • `availability_of()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입: 이메일 중복 확인과 탈퇴 후 3일 규칙 (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	18 `release_withdrawn()` · 20 `check_nickname()`

1. 하는 일

이 이메일로 **새로 가입할 수 있는지** 알려 줍니다.

DB에서 그 이메일을 가진 행	결과	join.html 이메일 칸 아래 문구
없음	`{"available": true}`	✓ Available
활동 중(`ACTIVE`)이거나 정지된(`SUSPENDED`) 회원	`{"available": false, "reason": "TAKEN"}`	This email is already in use.
탈퇴한 지 **3일이 안 됨**	`{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "2026-10-09T03:00:00Z"}`	You can sign up again after Oct 9.
탈퇴한 지 3일이 지남	`{"available": true}` (가입할 때 18번으로 비켜 둡니다)	✓ Available

함수	역할
`check_email()`	이메일로 행을 찾습니다
`availability_of()`	찾은 행을 보고 위 표대로 판단합니다. 20번 `check_nickname()`도 같은 판단을 씁니다

2. 언제 쓰나

부르는 곳	`release`	하는 일
`GET /auth/check-email` (24): join.html 이메일 칸의 **Check** 버튼	`False`	확인만 하고, DB는 바꾸지 않습니다
21 `signup()`: **Sign Up** 버튼	`True`	3일이 지난 탈퇴 행이면 18번으로 비켜 둔 뒤 가입합니다

3일 규칙이 있는 이유 (FR-108, 2026-10-03 결정): 탈퇴 직후 다른 사람이 같은 이메일이나 닉네임으로 바로 가입하면, 탈퇴한 사람인 척할 수 있습니다. 그래서 3일 동안은 그 값을 쓰지 못하게 막습니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta
```

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **`datetime` 줄과 `select`, `UserStatus`를 이번에 추가했습니다.** 이 두 함수는 `SignupRequest`를 뺀 나머지를 모두 사용합니다.

import	가져오는 것	쓰는 곳
`UTC`, `datetime`, `timedelta` (파이썬 기본)	시간대, 시각, 기간	지금 시각, 탈퇴 시각 + 3일
`select`	SQL `SELECT`를 파이썬으로 씁니다	이메일로 행 찾기
`UserStatus`	08번의 회원 상태 목록	`DELETED`인지 확인
`AvailabilityResponse`	결과의 모양 (`schemas/auth.py`)	돌려주는 값

함수

```
REJOIN_WAIT_DAYS = 3


def check_email(db: Session, email: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    return availability_of(db, user, release)


def availability_of(db: Session, user: User | None, release: bool) -> AvailabilityResponse:
    if user is None:
        return AvailabilityResponse(available=True)
    if user.status != UserStatus.DELETED:
        return AvailabilityResponse(available=False, reason="TAKEN")
    if user.deleted_at is not None:
        available_at = user.deleted_at + timedelta(days=REJOIN_WAIT_DAYS)
        now = datetime.now(UTC).replace(tzinfo=None)  # DB 의 시각은 UTC · 시간대 정보 없음
        if now < available_at:
            return AvailabilityResponse(
                available=False,
                reason="REJOIN_TOO_SOON",
                available_at=available_at.replace(tzinfo=UTC),
            )
    if release:
        release_withdrawn(db, user)
    return AvailabilityResponse(available=True)
```

문법 풀이

import 문법	뜻
`from 모듈 import A, B, C`	한 모듈에서 여러 이름을 한 줄로 가져옵니다

식	뜻
`release: bool = False`	기본값이 있는 인자입니다. 넘기지 않으면 `False`(중복 확인 API)이고, 가입할 때는 `release=True`로 부릅니다

식	뜻
`select(User).where(User.email == …)`	`SELECT * FROM users WHERE email = …` 입니다
`db.scalar(…)`	결과 **한 행**을 `User`로 가져오고, 없으면 `None`입니다 (`email`은 `UNIQUE`라서 많아야 1행입니다)
`User | None`	회원 또는 `None`입니다
`timedelta(days=3)`	3일이라는 기간입니다. 시각에 더하면 3일 뒤의 시각이 됩니다
`datetime.now(UTC).replace(tzinfo=None)`	지금 UTC 시각에서 시간대 표시를 뗍니다. DB에서 읽은 `deleted_at`에도 표시가 없어서, 둘을 비교하려면 모양을 맞춰야 합니다
`available_at.replace(tzinfo=UTC)`	화면에 보낼 때는 UTC 표시를 붙여서 JSON에 `…Z`로 나가게 합니다. 화면은 이것을 사용자 시간대로 바꿔 보여 줍니다

4. 흐름

1. 이메일을 17번으로 정리한 뒤, 그 이메일을 가진 행을 찾습니다.
2. 없으면 쓸 수 있다고 응답합니다.
3. 탈퇴한 회원이 아니면(활동 중이거나 정지) `TAKEN`으로 응답합니다.
4. 탈퇴했고 3일이 지나지 않았으면 `REJOIN_TOO_SOON`과 언제부터 쓸 수 있는지를 응답합니다.
5. 3일이 지났으면 (`release`가 참이면 18번으로 비켜 두고) 쓸 수 있다고 응답합니다.

5. 왜 이렇게

선택	이유
`release` 인자를 추가함 (템플릿에 없음)	중복 확인은 `GET` 요청입니다. `GET`은 "보기만" 하는 요청이라 DB를 바꾸면 안 됩니다 (새로 고침이나 미리 불러오기로 여러 번 불릴 수 있습니다). 실제로 비켜 두는 일은 가입할 때만 합니다
판단 부분을 `availability_of()`로 분리	닉네임(20)도 규칙이 같습니다. 한 곳만 고치면 둘 다 바뀝니다
정지(`SUSPENDED`)된 회원도 `TAKEN`	정지는 탈퇴가 아닙니다. 계정이 살아 있으니 이메일도 그 사람의 것입니다
`deleted_at`이 비어 있으면 3일이 지난 것으로 봄	정상이라면 탈퇴할 때 항상 기록됩니다(P1-09). 값이 없는 이상한 행 때문에 그 이메일이 영원히 막히지 않게 합니다
`available_at`을 UTC로 보냄	서버는 시각을 모두 UTC로 다룹니다(`docs/DB.md`). 한국 사용자와 미국 사용자 모두 자기 시간대로 볼 수 있습니다
이유를 `TAKEN`과 `REJOIN_TOO_SOON`으로 나눔	화면이 서로 다른 문구를 보여 줄 수 있습니다. "이미 사용 중"과 "며칠 뒤 가능"은 사용자가 할 일이 다르기 때문입니다

참고: 이 확인은 "그 이메일이 가입되어 있는지"를 누구에게나 알려 줍니다(이메일 존재 확인). 실시간 중복 확인 기능(FR-101)을 만들면 피할 수 없는 성질이라, 대부분의 사이트도 같은 방식을 씁니다. 마구 묻는 것을 막는 요청 횟수 제한은, P1-12의 제한 방식을 함께 쓰는 방안을 그때 검토합니다.

6. 시험 (`tests/test_auth_service.py`에 8개를 추가해서 모두 19개, 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 이메일	`available=True`
활동 회원, 정지 회원 (2개)	`TAKEN`
대소문자와 공백을 섞어서 물음	`" Taken@Example.COM "`도 `TAKEN`입니다 (17번으로 정리한 뒤 비교)
탈퇴한 지 2일	`REJOIN_TOO_SOON`이고, `available_at`은 탈퇴 시각 + 3일이며, UTC 표시가 붙어 있습니다
탈퇴한 지 3일 1시간, 확인만 함	`available=True`이고, 이메일은 **그대로**입니다
탈퇴한 지 4일, `release=True`	`available=True`이고, 이메일이 `deleted+…`로 바뀌며, 같은 이메일로 바로 가입됩니다
응답 모양	`{"available": true, "reason": null, "available_at": null}`

전체 시험 127개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.